

Object Oriented Programming with Java

Week-2 Theory points

- Constructor overloading: - when one class has multiple constructors then it is called Constructor overloading.

Syntax:

```
classname()
```

```
{  
}
```

Ex:

Class A

```
{
```

```
    A() // default
```

```
    {  
    }
```

```
    A(int a, int b) //parameterized
```

```
    {  
    }
```

```
}
```

- **Static variable:** - A **static variable** in Java, also known as a **class variable**, is a variable that is shared among all instances (objects) of a class.
 - Declared using the static keyword inside a class, but outside any method, constructor, or block.
 - Memory is allocated only once when the class is loaded into the Java Virtual Machine (JVM)
 - Can be accessed directly using the class name (ClassName.variableName), without needing to create an object of the class.
 - Changes made to a static variable by any instance are immediately visible to all other instances of the class.
 - static variables are automatically assigned default values (e.g., 0 for int, false for boolean, null for object references) if not explicitly initialized.

- When to Use Static Variables?
 - ✓ **For Common Properties:** When a property is common to all objects of a class, such as a company name for all employees or a college name for all students.
 - ✓ **For Counters:** To keep track of the number of objects created for a class.
 - ✓ **For Constants:** To define constants using the `static` `final` keywords (conventionally named in `UPPER_CASE`), which are fixed values shared by all instances.
 - ✓ **Memory Efficiency:** They save memory because a single copy is maintained at the class level, rather than multiple copies per instance.
- **Static method:** - A **static method** in Java is a method that **belongs to the class itself**, rather than to any specific instance (object) of that class. This means it can be called directly using the class name, without needing to create an object.
 - They can be accessed even if no objects of the class have been created. A prime example is the `main()` method, which is static so the Java Virtual Machine (JVM) can call it to start the program without instantiating an object.
 - Static methods can directly access only static variables and other static methods of the class. They **cannot directly access** instance (non-static) variables or methods.
 - The `this` and `super` keywords cannot be used within a static method.
 - Memory for static methods is allocated once when the class is loaded into memory, and they are shared by all instances.
- **this keyword:**-The `this` keyword in Java is a reference variable that refers to the **current object** (instance) within a class. It is used to access members of the current object and avoid naming conflicts.

```
class Student {
```

```

    int age; // Instance variable
    void setAge(int age) { // 'age' is a local parameter
        this.age = age; // 'this.age' refers to the instance variable
    }
}

```

- **Outer class & inner class:** an outer class is a top-level class that can contain other classes, while an inner class (a non-static nested class) is defined within the outer class and has a strong association with an instance of its container.

➤ Syntax:

```
OuterClass outerObject = new OuterClass();
```

```
OuterClass.InnerClass innerObject = outerObject.new InnerClass();
```

Ex:

Class A

```

{
    Void outer()
    {
    }
}

```

Class B

```

{
    Void inner()
    {
    }
}

```

Main()

```
{
```

```
A a= new A(); //outer class object
A.B b= a.new B(); //inner class object
}
}
```